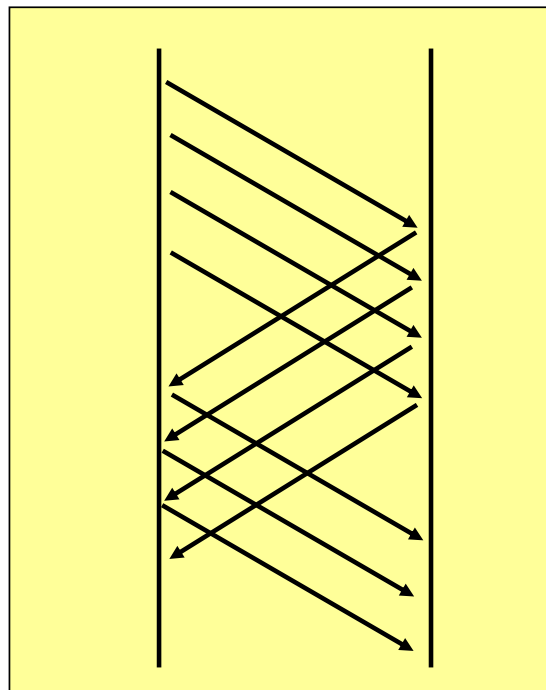




Die Idee der Sliding-Window-Protokolle





Pipelining zur Verbesserung der Übertragung

- Segmente werden auf Sender- und Empfängerseite gepuffert
 - Sender muss ein Segment vorhalten, bis es bestätigt wurde
 - **Empfänger muss ein Segment speichern, bis die Anwendung es ausliest und verarbeitet**
 - Senden aller Segmente ohne Bestätigung nicht sinnvoll:
 - > Große Puffer im Betriebssystem nötig
 - > Netz wird spontan sehr stark belastet
- *Prinzip der Flusskontrolle*
 - Erlaube das Senden mehrerer Segmente ohne Quittierung
 - Mit jeder regulär empfangenen Bestätigung können weitere Daten verschickt werden
 - Die Anzahl der ohne Quittierung ausstehenden Daten wird durch das Sliding-Window beschrieben



Konkreter

- Verwendung eines „Fensters“ der Größe W (in Segmenten oder in Bytes)
 - Anschaulich wird das Fenster logisch über den Stream gelegt
 - Es bestimmt, welche Daten des Streams versendet werden dürfen ohne dass auf eine Bestätigung gewartet werden muss
 - Wurden alle Daten „unter“ dem Fenster versendet, so ist vor dem Versenden weiterer Daten eine Bestätigung am „linken“ Rand des Fensters erforderlich
 - Kommt eine solche Bestätigung, so verschiebt sich anschaulich das Fenster nach rechts
 - Es dürfen somit weitere Daten versendet werden
 - Faktisch verschiebt sich das Fenster also mit jeder empfangenen Bestätigung nach rechts
 - Somit findet eine Rückkopplung statt: Bis zum Rand des Fensters kann man so schnell versenden wie man möchte, aber wenn man das Fenster ausgenutzt hat, so darf man erst etwas versenden, wenn der Empfänger es „verarbeitet“ hat





Konkreter

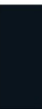
- Sender und Empfänger nutzen ein *Übertragungsfenster* (typischerweise die Größe des freien Pufferspeichers im OS beim Sender/Empfänger)
- Sender kennzeichnet die zu versendenden Bytes/Segmente des Datenstroms z.B. mit Nummerierung 0, 1,..., MODULUS-1, 0, ...
Es muss $W < \text{MODULUS}$ gelten
 - Hinweis: In der **Realität** wird der **Byte-Offset** im Stream verwendet, die **Fenstergröße bemisst sich somit in Bytes**
- Fenstergröße W bedeutet: der Sender darf maximal W fortlaufend nummerierte Segmente verschicken, ohne eine Bestätigung zu bekommen
- Empfänger bestätigt empfangene Segmente durch Quittungen (ACK)
- Sender „rückt“ Fenster um die Anzahl der Bytes des Segments vor, sobald ein ACK eintrifft, und darf somit weitere Segmente verschicken, bis er blockieren und auf ACKs warten würde





Das Prinzip der Sliding-Window-Protokolle

- Jedes Segment wird durch eine im Transport-Header gespeicherte Sequenznummer durchlaufend gekennzeichnet
 - Zyklische Nummerierung der Segmente
(z.B. 0, 1, 2, ..., $MODULUS-1$, 0, ...; wobei $W < MODULUS$)
 - **In der Realität wird die Position des ersten Bytes des Segments im Stream genommen
Dies vereinfacht die Handhabung variabel langer Segmente als das Durchnummerieren**
 - Damit gibt es die Fenstergrenzen MIN, MAX mit
 - $MAX - MIN = W$ für $MIN < MAX$ und
 - $MODULUS - MIN + MAX + 1 = W$ für $MIN > MAX$
- Empfänger bestätigt den korrekten Empfang durch Quittungen (ACK)
- ACKs beziehen sich auf eine Sequenznummer
 - Nummer des empfangenen Segments
 - Nummer des nächsten erwarteten Segments im Stream
 - **In der Realität: Offset des nächsten erwartetes Segment im Stream**
- Übertragungsfenster des Senders wird bei Empfang eines ACK angepasst (MIN und MAX werden angepasst), was faktisch ein Verschieben des Fensters bedeutet



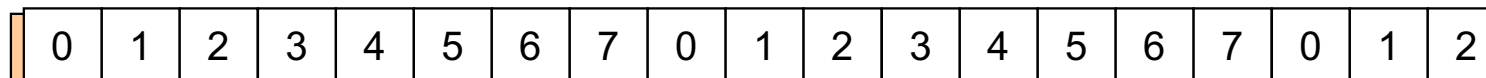


Sliding-Window-Protokoll

Beispiel (3-Bit Sende-/Empfangsnummer – MODULUS = $W+1$)

- Der Einfachheit halber nummerieren wir Segmente durch, ACKs beziehen sich auf Segmente
- Hier Sequenznummern mit 3 Bits: $m = 8$ mögliche Kombinationen
- Übertragungsfenster muss **kleiner** sein, z.B. können die Stationen eine Fenstergröße $W=7$ mit $1 \leq W < m$ vereinbaren
- **Das Fenster beschränkt die Anzahl der unbestätigten Byte, die zu einem Zeitpunkt unterwegs sein dürfen** (hier max. 7 wegen $W = 7$)
- Bei Empfang einer Quittung wird das Fenster entsprechend verschoben
- Bytes werden fortlaufend mod m nummeriert (für $m = 8$ also von 0 bis 7)

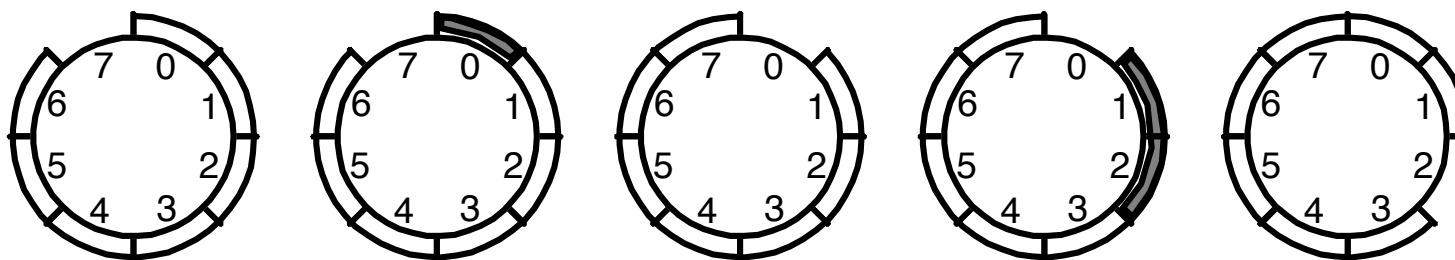
ACK ~~ACK~~ ACK 3



Zeit



Sliding Window - andere Darstellung





Wie groß sollte das Übertragungsfenster sein?

Bandwidth-Delay-Produkt als fundamentales Gesetz!

- Sei w die Größe des Übertragungsfenster in **Bits**:
 - Wir übertragen **maximal ein Fenster pro RoundTrip**
 - Erreichbarer Durchsatz $\leq w / \text{RTT}$
 - Bandwidth-Delay-Produkt: $w \geq \text{Bandwidth} * \text{Delay}$

Beispiel 1:

- Die Anwendung möchte eine Rate von 10Mbps erreichen. Wir kennen die Round-Trip-Zeit, die bei 8ms liegt. Die Segmente seien 500 Byte groß. Wie groß muss w sein?
 - > Antwort: $w \geq 10 \text{ Mb/s} * 0,008\text{s} = 80.000 \text{ Bit} = 10.000 \text{ Byte}$
 - > In Segmenten ausgedrückt: 20 Segmente a 500 Byte



Wie groß sollte das Übertragungsfenster sein?

Das Bandwidth-Delay-Produkt

Sei w die Größe des Übertragungsfenster in **Bits**:

- Wir übertragen maximal ein Fenster pro RoundTrip
- Erreichbarer Durchsatz $\leq w / \text{RTT}$
- Bandwidth-Delay-Produkt: $w \geq \text{Bandwidth} * \text{Delay}$

Beispiel 2:

- Die Anwendung möchte eine Rate von 1Gbps erreichen. Wir kennen die Round-Trip-Zeit, die bei 240ms liegt. Die Segmente seien 1000 Byte groß. Wie groß muss w sein?
- > Antwort: $w \geq 1 \text{ Gb/s} * 0,24\text{s} = 240.000.000 \text{ Bit} = 30.000.000 \text{ Byte}$
- > In Segmenten ausgedrückt: 30.000 Segmente a 1.000 Byte



Merke:

Sliding-Window-Protokolle sind der übliche Weg um Daten zuverlässig über einen unzuverlässigen Kanal zu übertragen. Der Einsatz dieser Protokolle hat aber fundamentale Auswirkungen auf die erzielbare Bandbreite.

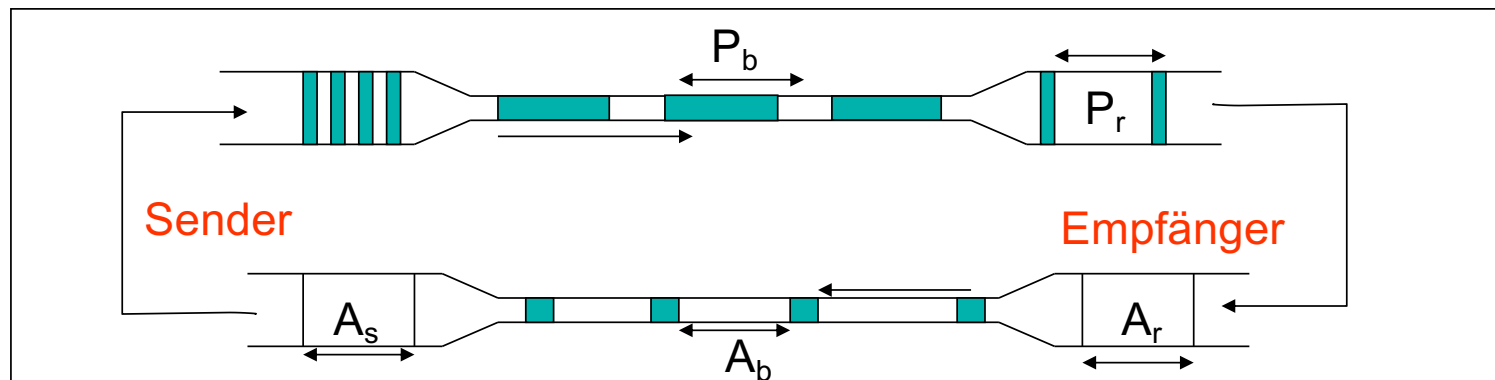
Das Bandwidth-Delay-Produkt grenzt die erzielbare Übertragungsrate unabhängig von der eigentlich verfügbaren Bandbreite des Netzes ein. Faktisch kann man nie schneller kommunizieren, als ein "Fenster" pro Round-Trip-Time.

Um unnötige Wartezeiten zu vermeiden und die Kapazitäten des Netzes zu nutzen muss demnach die Fenstergröße der Round-Trip-Zeit und der erwünschten Bandbreite angepasst werden.

Dies ist insbesondere für „Long-Fat-Pipes“ eine Herausforderung

Warum eigentlich Flusskontrolle?

- Wenn der Sender an die Grenze des Windows stößt, so darf er erst weiter senden, wenn ein ACK empfangen wird
- Feedback-Informationen zwischen Empfänger und Sender reguliert die Senderate der Quelle
- Der Fortschritt des Sliding-Windows passt sich den Fähigkeiten des Netzes an
- Im „*steady state*“ wird immer dann ein Paket verschickt, wenn ein ACK empfangen wird





Verfahren bei Fehler- und Sonderfällen

Welche Ereignisse können den geschilderten
Ablauf stören?





Lazy-Read

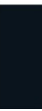
- Stellen Sie sich vor, dass die Daten erfolgreich beim Empfänger ankommen, jedoch die entsprechende Anwendung die Daten nicht liest
- Welche Probleme können auftreten und wie kann man mit diesen umgehen?





Fehlerereignisse beim Empfänger

- Empfang eines out-of-order-Segments
 - Eigentlich wurde ein anderes Segment erwartet. Schicken wir nun eine Bestätigung und wenn ja welche? Es wurde ja etwas empfangen...
 - Das Ignorieren eines nicht erwarteten Segments ist möglich, es wird dann verworfen. Allerdings bedeutet dies, dass dieses Segment wohl noch mindestens einmal erneut übertragen werden muss
 - Auch eine Zwischenspeicherung ist möglich. Was signalisieren wir hier dem Sender? Wenn wir speichern, sollte aber auch der noch verbleibende freie Speicherplatz im Puffer des Empfängers mit in das Verfahren einbezogen werden, sonst läuft der Puffer ggf. voll
- Empfang eines fehlerbehafteten Segments
 - Keine Reaktion (Der Sender wird irgendwann eine erneute Übertragung vornehmen, da ja keine Bestätigung verschickt wurde)
 - Falls unterstützt: Verschicken einer „negativen“ Bestätigung oder einer Nachricht, die dies kommuniziert (als Indiz für einen möglichen Paketverlust).
 - Wir können auch duplizierte kumulative Bestätigungen als NACK-Ersatz nutzen:
Ich warte noch immer auf Byte Nummer_x





Fehlerereignisse beim Empfänger

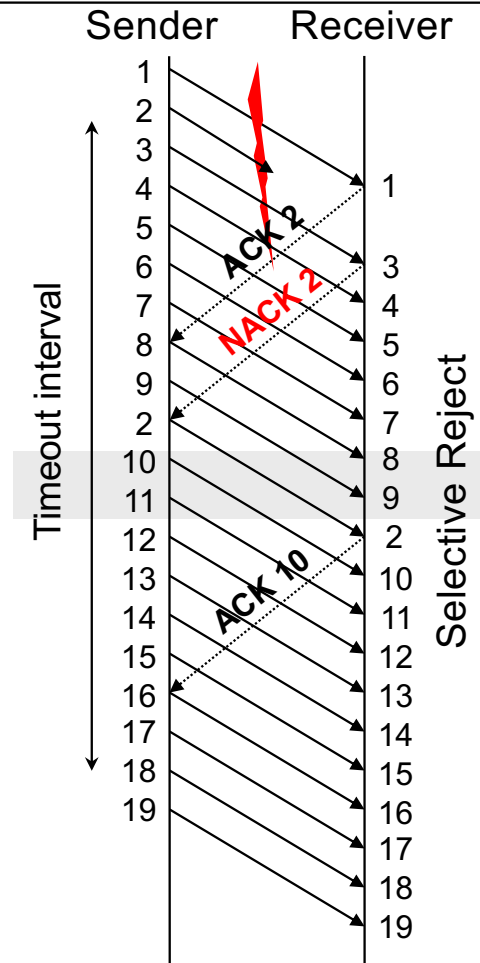
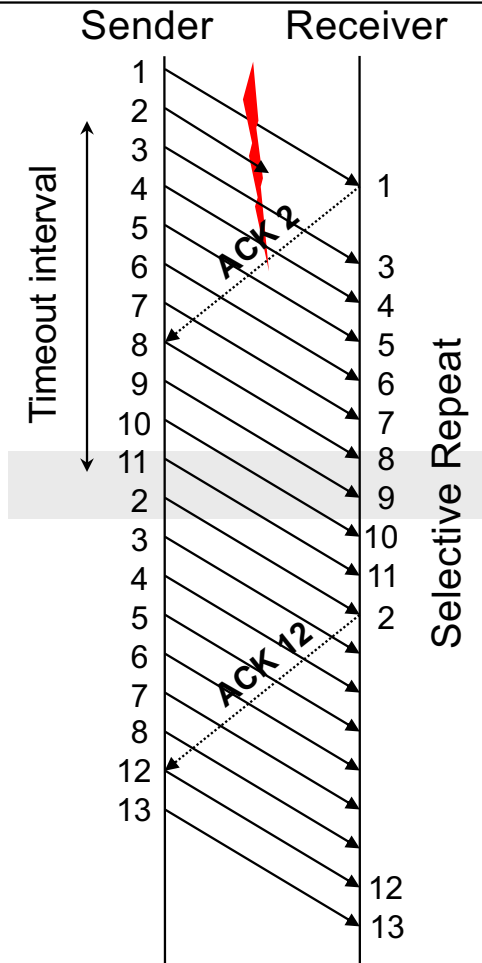
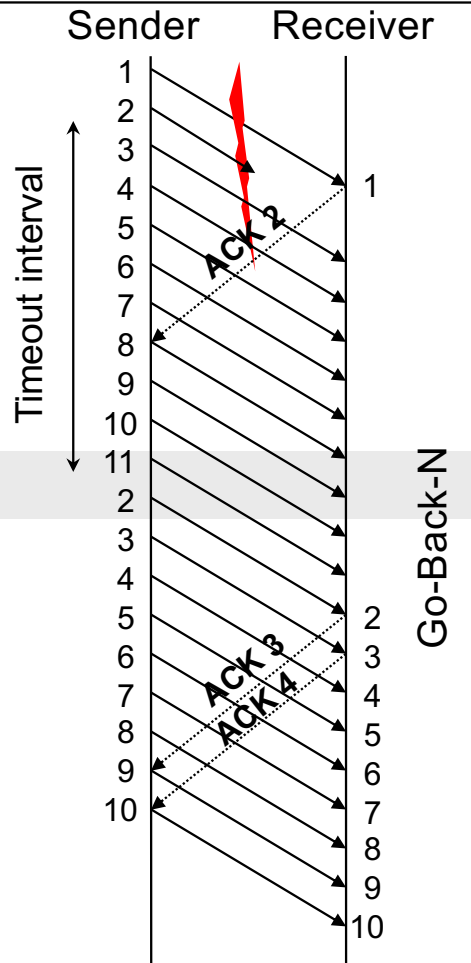
- Timeout -> Auf welches ausstehende Segment bezieht sich dieser?
Immer das zuerst gesendete?
- Empfang einer negativen Bestätigung(falls es die gibt)
- Ist der Empfang einer nicht erwarteten Bestätigung (out-of-order) ACKs überhaupt möglich?
- Reaktion

Identifikation und erneutes Verschicken des Segments und ggf. folgender

- (**Go-Back-N**): Wir nutzen keine Kenntnis über etwaig bereits empfangene Segmente. Somit übertragen wir ab dem Fehlerfall alles neu; **Empfänger ignoriert nicht erwartete Segmente**
- (**Selective Repeat**): Mittelbare Kenntnis über etwaig bereits empfangene Segmente, **Empfänger puffert nicht erwartete Segmente**, so dass sich bei Neuübertragung der „Lücke“ das Fenster ruckartig verschiebt
- (**Selective Reject**): Unmittelbare Kenntnis über unerwartete Ereignisse und sofortige Reaktion zur Behebung
- Anmerkung: Alle Segmente müssen bis zum Empfang einer Bestätigung beim Sender im Betriebssystem gespeichert bleiben

Fehlerbehebung Flusskontrolle

(Achtung: Die Größe des Windows kann ggf. zu einer vorübergehenden Blockade beim Sender führen)
 Fenstergröße mind. 10 Fenstergröße mind. 10 Fenstergröße mind. 15





Kumulative Bestätigungen als Kompromis

Idee der kumulativen Bestätigungen:

Eine Bestätigung zeigt an, dass alle Daten bis zu der verwendeten Nummer/Byte Position korrekt empfangen wurden

Kommt ein Out-of-Order-Segment an, so wird demnach eine Bestätigung verschickt, die schon einmal verschickt wurde
(**„ich warte immer noch auf Byte Nummer_x“**)

Speichert der Empfänger außerhalb der Reihenfolge empfangene Segmente zwischen, dann bedeutet ein „Lückenschluss“, dass dieser einen „Sprung“ bei den Bestätigungen bedingt





Sliding-Window-Protokoll mit kumulativen ACKs



Beispiel (3-Bit Sende-/Empfangsnummer mit MODULUS = $W+1$)

- Bestätigungen werden kumuliert verschickt werden
- Bei Empfang einer Quittung wird das Fenster entsprechend verschoben
- Bytes werden fortlaufend modulo m nummeriert (für $m = 8$ also Nummern von 0 bis 7)

ACK 1 ACK 3

ACK 7

0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7	0	1	2
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---



Kumulative Bestätigungen als Ersatz für NACKs

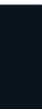
Wenn wir unser Verfahren konsequent umsetzen gilt:

1. Kommt ein Out-of-Order-Segment an, so wird eine Bestätigung verschickt, die schon einmal verschickt wurde ("ich warte immer noch auf Byte Nummer_x")
2. Wenn ein Segment verloren gegangen ist, dann werden alle weiteren empfangenen Segmente durch die gleiche Bestätigung quittiert. Diese zeigt auf das verloren gegangene Segment

Duplicate Acknowledgments: Empfang einer Bestätigung, die schon vorher erhalten wurde: ("ich warte immer noch auf x")

Segmente können sich überholen, deshalb sollte das nicht sofort als Fall 2 betrachtet werden (NACK)

Aber: **Triple Duplicate Acknowledgement wird als NACK aufgefasst!**





Aufpassen: Platz im Zahlenraum der Sequenznummern

Beispiel:

Sequenznummern: 0, 1, 2, 3

Fenstergröße = 3

Empfänger sieht keinen Unterschied zwischen den beiden Szenarios!
Fehlerhaftes Versenden neuer Daten in a

